# Lecture 2: Reducibility and Diagonalisation

Lecturer: Ari

**Notation**: We denote the complement of a set $A$ with a line above the set $\overline{A}$.

## 1. Introduction

We begin by re-stating the definitions of decidability and recursive enumerability, discussed in the previous class. Then we show that there are some languages that are not in RE. We do so using the diagonalisation argument, which we will use repeatedly throughout the class.

**Definition 2.1 (Recursively Enumerable).** We say a language $L \subseteq \Sigma^*$ is recursively enumerable (RE) if there exists a TM $M$ such that, for all $x \in \Sigma^*$, $M$ accepts $x$ iff $x \in L$.

**Definition 2.2 (Decidable Languages).** We say a language $L \subseteq \Sigma^*$ is decidable if there exists a TM $M$ such that

1. For all $x \in L$, $M$ accepts $x$.
2. For all $x \notin L$, $M$ halts and rejects $x$.

The main difference between the two is that for decidable languages, $M$ *always* terminates, whereas for RE languages, $M$ is only guaranteed to terminate for YES instances. For NO instances the TM could halt or go on forever, we have no guarantees. Clearly, every decidable language is also recursively enumerable. However, we will soon show that there are recursively enumerable languages that are not decidable. First we show the set of languages is uncountable.

**Lemma 2.3.** *Let $\Sigma$ denote some finite input alphabet. The set of languages over $\Sigma^*$ is uncountable.*

*Proof.* Assume for the sake of contradiction that this was not the case, and we could enumerate all the languages $L_1, L_2, \ldots, L_n, \ldots$ as shown in the rows of Figure 1. Let the columns of Figure 1 denote all the strings in $\Sigma^*$ arranged in lexicographic order.

| | $s_0$ | $s_1$ | $s_2$ | $\cdots$ | $s_n$ | |
|---|---|---|---|---|---|---|
| $L_0$ | 1 | 0 | 1 | $\cdots$ | 0 | |
| $L_1$ | 1 | 0 | 1 | $\cdots$ | 1 | |
| $L_2$ | 0 | 1 | 0 | $\cdots$ | 0 | |
| $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ | $\ddots$ | $\vdots$ | |
| $L_n$ | 0 | 0 | 1 | $\cdots$ | 1 | |
| $L'$ | 0 | 1 | 1 | $\cdots$ | 0 | flipped diagonal |

Figure 1: Diagonalisation argument for showing that the set of languages over a finite alphabet is uncountable.

Let $A[i, j] = 1$ if the string $s_j$ is present in $L_i$. Now construct a new language $L'$ by flipping the bits on the diagonal of $A$, and then gathering all strings where the inverted diagonal cell in

the column of holds a value of 1. So if $A[0,0] = 1$, $L_0$ contains string $s_0$, but $s_0 \not\in L'$. So $L'$ is already different from $L_0$. $L'$ is a set with a special property: it differs from every language $L_i$ listed in table $A$. For every $L_i$, if $L_i$ contains $s_i$, $L'$ does not, and vice versa. Thus $L'$ could not have been listed in the table, and thus the set of languages is uncountable. □

The way we proved Lemma 2.3 is known as the *diagonalisation* element. We will use this argument all the time. Next, we show that there are languages that are *not even* recursively enumerable.

**Theorem 2.4.** *The exists a language $L \subseteq \Sigma^*$ such that $L \notin \mathsf{RE}$.*

*Proof.* A TM is described by the tuple

$$(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}),$$

where $Q$, $\Sigma$, $\Gamma$ are finite sets, $\delta$ is a finite transition table, and $q_0$, $q_{\text{accept}}$, $q_{\text{reject}}$ are designated states in $Q$. Thus, the description of a TM is finite. Fix a binary encoding that gives each TM a unique description. List all valid descriptions by increasing length, breaking ties lexicographically. This lists every TM exactly once as $M_0, M_1, M_2, \ldots$, so the set of TMs is countable. By Lemma 2.3, the set of languages over a finite alphabet is uncountable. For every language to be in $\mathsf{RE}$, we would need a different[1] TM to recognise each language. This is impossible, as the set of TM's is countable but the set of languages is uncountable. □

Next we show that there are languages that are RE but not decidable. Before we do, we recap the idea of a universal Turing machine. Remember from Lecture 01, that a TM consists of a 7 tuple $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$. Each element in the tuple is a set of finite size. Thus, we can encode any TM $M$ with a binary string $\lfloor M \rfloor \in \{0,1\}^*$. The universal TM $\mathcal{U}$ is a TM, that takes as input a string $x \in \{0,1\}^*$, and an encoding of a machine in binary $\lfloor M \rfloor \in \{0,1\}^*$, and outputs $\mathcal{U}(x, \lfloor M \rfloor) = M(x)$. It can be shown [AB09, Theorem 1.13] that if $M$ halts on input $x$ within $T$ steps then $\mathcal{U}(x, \alpha)$ halts within $CT \log T$ steps, where $C$ is a number independent of $|x|$ and depending only on $M$'s alphabet size, number of tapes, and number of states.

**Theorem 2.5.** *The exists a language $L \subseteq \Sigma^*$ such that $L \notin \mathsf{Decidable}$ but $L \in \mathsf{RE}$.*

*Proof.* Define

$$\mathsf{Accept}_{\mathsf{TM}} = \{(\lfloor M \rfloor, x) : M \text{ is a TM}, \ x \in \Sigma^*, \ M \text{ accepts } x\}.$$

Here each pair $(\lfloor M \rfloor, x)$ is encoded as a binary string. As $\mathsf{Accept}_{\mathsf{TM}}$ is a set of strings it is a valid language, and we want to show that

$$\mathsf{Accept}_{\mathsf{TM}} \in \mathsf{RE} \wedge \mathsf{Accept}_{\mathsf{TM}} \notin \mathsf{Decidable}$$

Clearly $\mathsf{Accept}_{\mathsf{TM}} \in \mathsf{RE}$, as given any pair $(\lfloor M \rfloor, x)$ we can use the universal TM to simulate $M$ on $x$, and by definition the universal TM $\mathcal{U}$ accepts *if and only if* $M$ accepts $x$.

Now to show $\mathsf{Accept}_{\mathsf{TM}} \notin \mathsf{Decidable}$ we once again draw a table as shown in Figure 2.

Let the rows of Figure 2 be indexed by strings $s_i$, with row $i$ representing the TM $M_{s_i}$ encoded by $s_i$ (an invalid encoding is interpreted as a TM that always rejects). The columns

[1] We need different TM's because any given TM $M$ can only recognise exactly one language. That language $L_M$ is the set of strings accepted by $M$. Two different TM's $M$ and $M$' could accept the same language $L = L_M = L_{M'}$, but two different languages $L \neq L'$, but would need at least two recognisers.

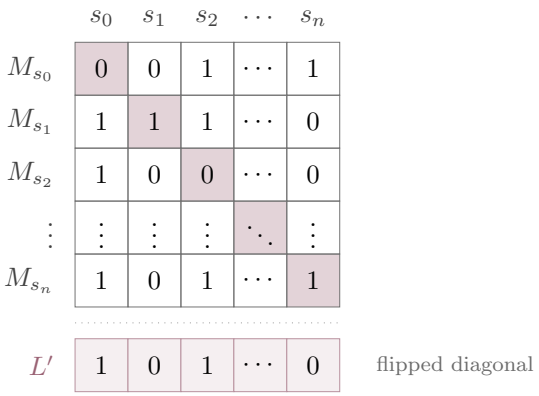



Figure 2: Diagonalisation argument for showing there are RE languages that are undecideable.

represent the same strings $s_j$, arranged by increasing length and then lexicographically. We set $A[i,j] = 1$ *iff* $M_{s_i}$ accepts $s_j$, and $A[i,j] = 0$ otherwise.

Construct a new language $L'$ such that $s_i \in L' \iff M_{s_i}$ does not accept $s_i$. In other words, $L'$ consists of those strings $s_i$ for which $A[i,i] = 0$.

Assume that $\mathsf{Accept}_{\mathsf{TM}}$ is decidable, and let $D$ be a decider for it. Then $L'$ is also decidable: on input $s_i$, run $D$ on $(\lfloor M_{s_i} \rfloor, s_i)$ and reverse its answer. Since $D$ always halts, this procedure always halts and accepts exactly when $M_{s_i}$ does not accept $s_i$.

Decidability implies that there exists some TM $M$ listed in the table in Figure 2 (as we listed all the TMs), let's say the one encoded by $s_j$, that recognises $L'$, i.e. $L' = L_{M_{s_j}}$, where $L_M$ denotes the set of strings accepted by a TM $M$. But $s_j \in L' \iff s_j \notin L_{M_{s_j}} \iff s_j \notin L'$, so we have a contradiction! □

Neither of the proofs above gives us a way to show that a specific language is not RE. Using the following lemma, we can come up with an explicit construction of a language that is not in RE.

**Lemma 2.6.** $L \in \mathsf{Decidable} \iff L \in \mathsf{RE} \wedge \overline{L} \in \mathsf{RE}$

*Proof.* Clearly if $L \in \mathsf{Decidable}$, $\overline{L} \in \mathsf{Decidable}$ as well, just by interchanging accepting and rejecting states of a TM deciding $L$, and hence both $L$ and $\overline{L}$ are in $\mathsf{RE}$.

For the converse, given recognizers $M$ for $L$ and $M'$ for $\overline{L}$, we design a decider $N$ for $L$ as follows. On input $x$, $N$ runs $M$ and $M'$ in parallel on $x$. If $M$ accepts on $x$, $N$ accepts; if $M'$ accepts on $x$, $N$ rejects. Note that either $M$ or $M'$ must eventually accept on $x$, as $M$ recognizes $L$ and $M'$ recognizes $\overline{L}$. □

This gives us the following corollary, that the set $\overline{\mathsf{Accept}_{\mathsf{TM}}}$ is not in RE. This is because if it was, then $\mathsf{Accept}_{\mathsf{TM}}$ would be decidable.

**Corollary 2.7.** $\overline{\mathsf{Accept}_{\mathsf{TM}}} \notin \mathsf{RE}$.

## 2. Reductions and Completeness

We not want to come up with a new diagonalisation argument every time we're asked to prove if a language is decidable or not. Instead, we use the idea of reductions. Intuitively, reductions allow us to compare the complexity of two languages (such as whether they're both computable or not). Informally, we say a language $L_1$ reduces to a language $L_2$, denoted by $L_1 \preceq L_2$, if $L_1$ is

at least as easy as $L_2$. In other words, if we can find a way to solve $L_2$ then we can solve $L_1$. Alternatively, we can also think of the above statement as saying $L_2$ is at least as hard as $L_1$. In this part of the course, we will use reductions to establish hardness. Specifically we will have some language $L_1$ that is hard, say undecidable. Then we will use that to show that $L_2$ is also undecidable, as solving $L_2$ is at least as hard as solving $L_1$ (could be harder).

## 2.1. Many-One/Mapping Reductions

**Definition 2.8.** A mapping/many-one reduction from language $L_1$ to language $L_2$ is a computable function $f : \Sigma^* \to \Sigma^*$ such that $w \in L_1$ if and only if $f(w) \in L_2$. In this case, we say $L_1$ is mapping/many-one reducible to $L_2$, and we denote it by $L_1 \preceq_m L_2$.

**Theorem 2.9.** *If $L_2$ is RE (resp. decidable) and $L_1 \preceq_m L_2$, then $L_1$ is also RE (resp. decidable).*

*Proof.* Given $x \in L_1$, compute $f(x)$, and then output whatever TM $M$ for $L_2$ outputs on $f(x)$. □

**Definition 2.10 (Halting Problem).** Define the language $\mathsf{Halt}$ as the

$$\mathsf{Halt} = \{(\lfloor M \rfloor, w) : M \text{ halts on input } w\}$$

The intuition behind the following proof is as follows: given a Turing machine $M$, we construct another Turing machine $M'$ such that $M$ accepts $w$ iff $M'$ halts on $w$.

**Lemma 2.11.** *[Sip96, Example 5.24]* $\mathsf{Accept}_{\mathsf{TM}} \preceq_m \mathsf{Halt}$

*Proof.* We are given $(\lfloor M \rfloor, w)$ and we need to construct $(\lfloor M' \rfloor, w')$ that is can be used as input to $M_H$. Define $f$ as follows: on input $(\lfloor M \rfloor, w)$:

1. Construct $\lfloor M' \rfloor$ for $M'$ which operates as follows on any general input $x$, ignoring $x$ and using the fixed $M$ and $w$ from the reduction's input:
   (a) Run $M$ on input $w$.
   (b) If $M$ accepts, output Accept.
   (c) If $M$ rejects, enter a loop.
2. Output $(\lfloor M' \rfloor, w)$.

□

**Definition 2.12 (Equivalence).** $\mathsf{EQ}_{\mathsf{TM}} = \{(\lfloor M_1 \rfloor, \lfloor M_2 \rfloor) : L_{M_1} = L_{M_2}\}$ where $L_M$ denotes the set of strings accepted by $M$.

**Lemma 2.13.** $\mathsf{Accept}_{\mathsf{TM}} \preceq_m \mathsf{EQ}_{\mathsf{TM}}$

*Proof.* On input $(\lfloor M \rfloor, x)$ construct encodings of the following TM's $M_1$ and $M_2$.

1. $M_1$ is a TM that accepts always.
2. $M_2$ runs $M$ on $x$ and if $M$ accepts, $M_2$ outputs accept.
3. Output $(\lfloor M_1 \rfloor, \lfloor M_2 \rfloor)$

Why does this work? If $M$ accepts $x$, then $(\lfloor M \rfloor, x) \in \mathsf{Accept_{TM}}$. We need to show that $(\lfloor M_1 \rfloor, \lfloor M_2 \rfloor) \in \mathsf{EQ_{TM}}$. Since $M$ accepts, then $M_2$ will always accept everything as well, and recognise the same set of strings as $M_1$. On the other hand, if $M$ does not accept $x$, then $M_2$ rejects all its inputs, and hence is not equivalent to $M_1$. □

These lemmas establish that $\overline{\mathsf{Halt}} \notin \mathsf{RE}$, and that $\mathsf{Halt}$ and $\mathsf{EQ_{TM}}$ are not decidable.

## 2.2. Completeness

Reductions give us a way to relate languages. If an undecidable language reduces to $L$, then $L$ is undecidable as well. We want more information. We want to know what the hardest $\mathsf{RE}$ language is. This is what we call $\mathsf{RE}$-completeness.

**Definition 2.14.** $L$ is $\mathsf{RE}$-complete if $L \in \mathsf{RE}$ and for all $L' \in \mathsf{RE}$, $L' \preceq_m L$.

It may be that there are no languages that are known to be complete. But in fact if can show that many of the well studied languages are $\mathsf{RE}$-complete.

**Lemma 2.15.** $\mathsf{Accept_{TM}}$ *is* $\mathsf{RE}$*-complete.*

*Proof.* If $L' \in \mathsf{RE}$, this implies that there exists some $M$ that recognises $L'$. Then the reduction $f$ takes $x$, and maps it to $(\lfloor M \rfloor, x)$ (as $M$ is fixed independent of $x$). $x \in L'$, then $M$ accepts $x$, so $(\lfloor M \rfloor, x) \in \mathsf{Accept_{TM}}$. □

**Lemma 2.16.** $\mathsf{Halt}$ *is* $\mathsf{RE}$*-complete.*

*Proof.* $\mathsf{Halt} \in \mathsf{RE}$ and $\mathsf{Accept_{TM}} \preceq_m \mathsf{Halt}$. So take any $L' \in \mathsf{RE}$, reduce to $\mathsf{Accept_{TM}}$ using the above transformation. Compose this reduction with the reduction from $\mathsf{Accept_{TM}}$ to $\mathsf{Halt}$ to obtain a reduction from $L'$ to $\mathsf{Halt}$. □

Is $\mathsf{EQ_{TM}}$-RE complete? No! Although, $\mathsf{Accept_{TM}}$ reduces to $\mathsf{EQ_{TM}}$, and it satisfies the hardness condition that every language in $\mathsf{RE}$ reduces to $\mathsf{EQ_{TM}}$, it's not clear that $\mathsf{EQ_{TM}}$ is in $\mathsf{RE}$. Intuitively, the reason $\mathsf{EQ_{TM}}$ is not in $\mathsf{RE}$ shows up in the quantifiers. Whether $M$ accepts $x$ *within* $t$ *steps* is easy to check: simulate $M$ on $x$ for $t$ steps. So membership in $\mathsf{Accept_{TM}}$ or $\mathsf{Halt}$ needs only a single $\exists$ as shown below

$$(\lfloor M \rfloor, x) \in \mathsf{Accept_{TM}} \iff \exists t \;\; M \text{ accepts } x \text{ within } t \text{ steps},$$
$$(\lfloor M \rfloor, x) \in \mathsf{Halt} \iff \exists t \;\; M \text{ halts on } x \text{ within } t \text{ steps}.$$

A TM recognises such a language by using $t = 1, 2, 3, \ldots$ steps and it accepts as soon as one works. If none works it runs forever, which $\mathsf{RE}$ allows.

$\mathsf{EQ_{TM}}$ says that $M_1$ and $M_2$ accept exactly the same inputs. So for every input $x$, one of two things happens: either both machines accept $x$, or neither machine ever accepts $x$ (it may reject, or loop forever). These two cases are very different. If a machine accepts $x$, it does so after some finite number of steps, so we can find out by waiting long enough. If a machine never accepts $x$, then however long we wait it has still not accepted, and no finite wait can confirm that it never will. In symbols, $(\lfloor M_1 \rfloor, \lfloor M_2 \rfloor) \in \mathsf{EQ_{TM}}$ if and only if

$$\forall x \Big[ (\exists t_1 \; M_1 \text{ accepts } x \text{ within } t_1 \text{ steps}) \wedge (\exists t_2 \; M_2 \text{ accepts } x \text{ within } t_2 \text{ steps})$$
$$\vee \forall t \; (M_1 \text{ does not accept } x \text{ within } t \text{ steps} \; \wedge \; M_2 \text{ does not accept } x \text{ within } t \text{ steps}) \Big].$$

The first line is the case where both accept. The second line is the case where neither does: for every $t$, neither $M_1$ nor $M_2$ accepts $x$ within $t$ steps. That is why we do not expect $\mathsf{EQ_{TM}}$ to be in $\mathsf{RE}$. Later on, we will consider the Polynomial Hierarchy, where quantification will be over strings of polynomial length in $x$, and observe an analogous phenomenon where some languages seem to require greater quantifier complexity than others.

*Remark.* A yes-instance of $\mathsf{EQ_{TM}}$ can be a pair of machines that never accept anything at all, for example two machines that reject every input. Then nothing ever happens: there is no accepting run to find and no step $t$ to point to. A recogniser watching them sees only silence, and silence never confirms a yes, since a machine that accepts something much later looks identical until the moment it does.

## References


[AB09] S. Arora and B. Barak. *Computational complexity: a modern approach*. Cambridge University Press, 2009.

[Sip96] M. Sipser. "Introduction to the Theory of Computation". In: *ACM Sigact News* 27.1 (1996), pp. 27–29.